TEAM 5 / GOLD MEMBER WEEK

EventHub

Maintenance of the Software Application

CEN 4020 AI Project — Part 6 | Call 6 | November 9, 2026

Jaideep Yarasingu — Gold member • John Bachta — teammate / reviewer

This report follows our Part 5 EventHub build. I treated the Call 6 incident as a case study, checked the actual code we had, then made changes that should prevent similar problems. I also kept the earlier project files and tests so John can review the new work without losing what he did in Part 5.

What this submission covers

Deliverable	What I prepared	Weight
1. Changes after Call 6	New EARS requirements, updated C4 component view, currency and safety changes, Spanish localization.	20%
2. Maintenance process	Actual refactoring decisions, incident analysis, before/after design, AI-assisted review, errors found in tests, recovery and monitoring procedures.	60%
3. Updated code + tests	Modular Flask/SQLite application, regression tests, backup and monitor utilities, original Part 5 system suite, reproducible commands.	20%

Final Part 6 verification — completed November 9, 2026

After the initial offline checks, I ran the completed Part 6 build with its required dependencies on my Mac. The final run passed 160 unit/integration tests and 139 Selenium browser/load tests: 299 passed, zero failed or skipped. Python compilation, the project Ruff lint and formatting checks, the EARS/C4 eligibility gate, and requirement-to-test traceability also passed. The Selenium run reported one urllib3/LibreSSL compatibility warning, not a failed test. The actual evidence is included in report/last_run.json, report/unit_run/results.json, report/system_run/results.json, and report/test_dashboard.html.

The older results remain part of the development record: John's Part 5 project reported 281 passing tests, while my first offline Part 6 check had 94 selected passes with dependency-related skips. Those earlier runs are superseded by the full 299-test Part 6 verification recorded on November 9. This proves the tested classroom build passed its configured checks; it does not establish a production deployment or resolve open operational decisions.

1. What changed after Call 6 (20%)

1.1 First, what the incident tells us

In the meeting, release 1.3 accidentally treated every currency as if it had two decimal places. That turned a ¥4,500 ticket into ¥45, and three students bought tickets for much less than intended. The team restored release 1.2 instead of debugging on the live site. The audit data helped explain what happened, but nobody was monitoring it to raise an early warning.

I checked our Part 5 code before making changes. Our `compute_quote()` already stores amounts in integer minor units and knows JPY has zero decimal places. So I did not claim to reproduce the fictional bug in our code. Instead, I left the price calculation alone, added explicit yen regression tests, and moved the new regional display code into a separate module that cannot write to orders.

1.2 Requirement and model updates

Call 6 request	New IDs	Implementation
Currency-safe display	pricing-018/019/020	Spanish and English display formatting isolated from the one price function; yen keeps zero decimal places.
Admin rate update test	admin-009	Next purchase uses new stored rate; previous purchase keeps its original rate.
Someone notices wrong totals	audit-009/010/011	Outlier scanner, persisted alert rows, admin acknowledgement, optional configured SMTP.
Spanish and regional dates	system-010/011	JSON translation catalog, language selector, localized event timestamps and currency presentation.
Backups and recovery	system-012/013	Consistent SQLite snapshot, SHA-256 integrity manifest, guarded restore operation.

The earlier IDs were kept. The additions bring the repo from 107 to 118 EARS requirements, and the original supersession links still pass the course checker. The Call 6 model adds Localization & Formatting, Order Anomaly Monitor, and Backup & Recovery components. `docs/part6-traceability.md` maps every change to the exact files and tests.

1.3 Customer decisions we did not invent

The transcript gives us the required direction, but not a numeric anomaly threshold, a named person to notify, a backup schedule or a full translation/time-zone policy. I kept those in Q22–Q26 and RISK-006–010. The demo uses a clearly provisional 25% anomaly threshold and does not claim anyone is on call.

2. The maintenance process (60%)

2.1 How I used the C4 diagram to guide changes

I described the new boundaries as small contracts: what a module receives, what it returns, what it is allowed to change, and what must remain true. That is more useful for maintaining the code than telling AI to “make the app cleaner.”

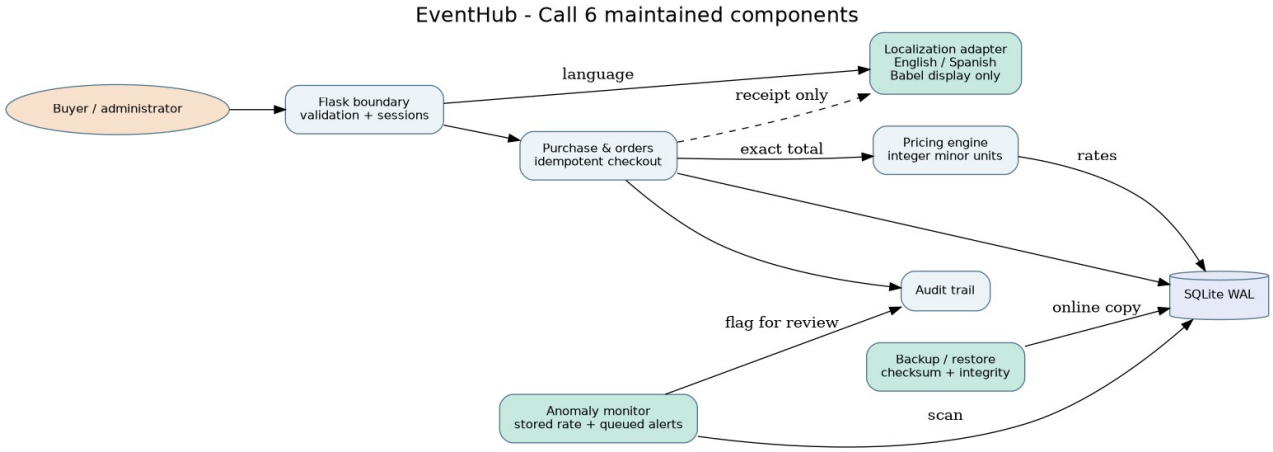


Figure 1. Updated component relationships. Green modules are new maintenance boundaries. The source architecture is in `workspace.dsl` and the drawing source is in `diagrams/Part6Components.dot`.

2.2 Changes that make future work easier

Before	After / reason
Pricing and display words were close to the same flow.	`compute_quote()` stays the only function that calculates the charged amount. `i18n.local_money()` is presentation-only and takes a finished integer amount.
English text lived in HTML templates.	English/Spanish JSON catalogs + a small adapter. A third language can be added without rewriting the order transaction.
Audit entries existed but no anomaly scanner did.	A separate scanner evaluates stored orders and queues one durable alert per order/reason. Notifications are outside the transaction.
WAL database had no project-level recovery tool.	SQLite online backup + integrity check, SHA-256 manifest and explicit offline restore confirmation.

2.3 One small refactor with a big safety benefit

The amount charged is still computed and stored as an integer. Changing what Spanish or English users see must not touch that value. Here is the important separation:

```
# Money calculation (still the single source of truth)
quote = compute_quote(base_usd_minor, quantity, eligible, currency, rate)
order_total_minor = quote.total_minor   # saved in SQLite

# Call 6: display only, after the charge is settled
shown_to_user = local_money(order_total_minor, currency, language)
# JPY remains 4500 integer yen, never 45.00 after formatting.
```

In the locale adapter I used Babel for Spanish display and `Decimal` to turn integer minor units into a text value. For English I kept our already-tested receipt strings like `JPY 4,500`, since changing that format would break older tests without improving the charge calculation. The Spanish page formats money and dates according to the selected locale.

2.4 How the AI-assisted changes were checked

I used ChatGPT to inspect the Part 5 source, propose the module boundaries, and draft code and regression tests. I checked these suggestions against the Call 6 transcript and the current project. I kept the real code and test output instead of claiming we saw the exact fictional production incident.

Step	What I checked or changed
Baseline	Inspected Part 5 pricing, exchange-rate edits, SQLite WAL, audit writes, templates, existing tests and 281-test dashboard.
Generate focused work	Built separate localization, monitor and backup modules; added core Spanish strings and two additional Selenium scenarios.
Review an AI-written test	My initial parameterized yen test wrongly asserted `JPY 4,500` for USD, EUR and GBP rows. That created three real failures.
Correct the test, not the code	The price function was correct. I narrowed the yen-specific display assertion to the JPY case, reran tests, and kept the error in the evidence notes.
Regression & handoff	Added persisted-purchase tests for every currency and an administrator rate-edit regression. Ran EARS/C4 and traceability checks, then fixed Python 3.9 compatibility and browser-test isolation/timing problems and reran the final suite. The completed Mac run passed 299 tests.

The main thing I learned is that a generated test can be wrong even when the application is right. I should check an assertion against the requirement and the input values before changing production code just to make a test green.

2.5 Audit monitoring and the incident response

An audit record that nobody reads is only useful after someone reports an incident. The new `monitor.scan_orders()` checks historic orders using each order's saved USD unit price, exchange rate and discount. It does not use today's rates, because that would incorrectly flag purchases made before an administrator changed a rate.

If a saved order is far from the expected amount, the monitor writes one row to `maintenance_alerts`. The administrator page lists the alerts and allows acknowledgement without deleting the evidence. `scripts/check_order_anomalies.py` can send a message using a configured SMTP server; without a recipient, it keeps the alert visible and exits with a warning. We cannot say this pages an on-call human until somebody configures a real monitoring schedule and owns the inbox.

```
# Key property: the scanner is idempotent
new_alerts = scan_orders(connection, deviation_percent=25)  # provisional
# For a deliberately damaged JPY test order: first scan = 1, next scan = 0
# Acknowledgement records the administrator; it does not erase the alert.
```

2.6 Recovery and rollback are different problems

Call 6 says the team rolled the program back to release 1.2 in 11 minutes. That is a story in the assignment, not a rollback I performed on a live host. I wrote the release runbook instead: save a known-good code version, back up the database, stop checkout if pricing is wrong, restore the known-good code, then verify exact amounts before making a new patch.

For the database, simply copying `eventhub.sqlite3` while WAL mode is active can miss committed data. The new tool uses SQLite's own backup API and writes a checksum manifest. Restore is explicitly offline, requires confirmation, verifies the checksum and validates the resulting SQLite database. The owner, backup schedule, retention and offsite destination still need a decision.

2.7 Adding Spanish without another pricing bug

The new language selector sets an English/Spanish preference. Main pages, purchase pages and receipt headings use the translated catalog, while event dates and money have locale-aware display in Spanish. The test checks that the original JPY amount is still 4,500 after formatting. Translations are student-written and should be reviewed with real users before a production release.

3. Updated code and regression evidence (20%)

3.1 Tests that directly address the incident

Scenario	Expected result
$30 USD ticket	Charge exactly 3,000 cents and display USD 30.00 in English.
Same ticket converted at 0.92 EUR/USD	Charge exactly 2,760 EUR cents.
Same ticket converted at 0.78 GBP/USD	Charge exactly 2,340 GBP pence.
Same ticket converted at 150 JPY/USD	Charge exactly 4,500 yen, no two-decimal division.
Admin edits EUR 0.92 to 1.10	Next $18 ticket charges 1,980 EUR cents; older order stays 1,656.
Tampered JPY order of 27 versus expected 2,700	One durable anomaly alert; rerunning scan makes no duplicate alert.
Backup checksum changed after creation	Offline restore rejects the backup and preserves the destination.
Spanish display	Core UI translations and date/currency display change; original integer charge does not.

3.2 Exactly what I executed

Check	Result in this environment
Python compileall	PASS on application, test and maintenance scripts.
Course EARS/C4 gate	PASS: 118 requirements, 14 supersession link ends, zero failures. Structurizr CLI not available.
Requirement-to-test mapping	PASS: 111 active requirements, 108 covered by tags, 3 with pre-existing exemptions.
Call 6 domain test file	PASS: 14 new Call 6 domain tests locally; all HTTP and unit cases subsequently passed in the final 160-test Mac unit/integration run.
Broader existing + new tests	Final Mac run: 160 passed, 0 failed or skipped. Earlier offline check of 94 passes with missing-dependency skips is retained as development history.
Previously supplied Part 5 result	281 passed (144 unit/integration, 137 browser/load) — archived Part 5 baseline only; not the Part 6 verdict.
New Selenium + full suite	Final Mac Part 6: 139 Selenium browser/load tests passed, 0 failed, 0 skipped; 1 urllib3/LibreSSL warning. With the 160 fast tests, total 299 passed. See report/last_run.json and report/test_dashboard.html.

The final Mac verification has now completed with all 299 unit, integration and Selenium tests passing. This includes the Part 6 regression scenarios alongside the inherited Part 5 tests. The initial offline limitations and subsequent test corrections are documented in the evidence files. John can review the current code, the generated dashboard, and the new requirements before the team submits the final version. The full updated Part 6 suite has not independently run in GitHub Actions.

4. Handoff to John and final maintenance decisions

4.1 Files John should review

File/folder	Why it matters
eventhub/i18n.py; locales/; templates/; static/	Core Spanish UI and display-only date/money formatting.
eventhub/monitor.py; scripts/check_order_anomalies.py	Alert logic, threshold, durable review queue and conditional SMTP.
eventhub/backups.py; scripts/backup_db.py; scripts/restore_db.py	Consistent WAL backups, checksums and an explicit restore guard.
features/*/requirements.md; workspace.dsl; diagrams/Part6Components.*	Changed requirements and revised model with old designs retained.
tests/test_call6_*; tests_system/test_call6_browser.py	New fast, HTTP and Selenium tests; check currency and admin workflows.
docs/release-and-recovery.md; docs/open-items.md; docs/risk-register.md	Rollback procedure and unresolved customer/operational choices.

4.2 Pending decisions, not hidden defects

- Q22: The 25% anomaly threshold is a demo assumption, not an approved production policy.
- Q23: An alert inbox/SMTP relay and a named on-call operator must be assigned; code alone will not notify anybody.
- Q24: English/Spanish translations and time-zone choices need review from the customer and international users.
- Q25: Assign backup ownership, frequency, offsite retention and perform a real recovery drill.
- Q26: Confirm the real deployment mechanism, restore a known-good release and document a timed rollback drill.

4.3 My recommendation for the team

Before submission, John should review the updated Part 6 package and its successful Mac test evidence, including the requirement and model changes. I kept the source and executed tests together so the delivered ZIP matches the verified build. Monitoring contact, backup ownership and schedule, and some customer policy decisions are still provisional. I would describe this as a verified classroom maintenance implementation, not a production deployment or a resolved operational handoff.

The big takeaway for me is that maintainability is not about changing a lot of code. It is about keeping responsibilities clear so we can fix one issue without breaking other features. I tried to do that by keeping pricing separate from translations, alerts separate from purchasing, and backups separate from the web app.